

Embedded Programming

Dr. Charis Kouzinopoulos

Department of Advanced Computing Sciences

Maastricht University

2024-2025

charis.kouzinopoulos@maastrichtuniversity.nl



Maastricht University

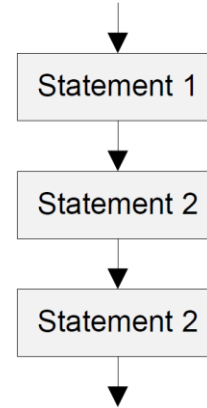
ARM assembly

Flow control



Three Control Structures

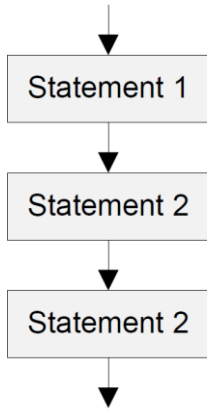
- Sequence Structure
 - Computer executes statements (instructions), one after another, in the order listed in the program



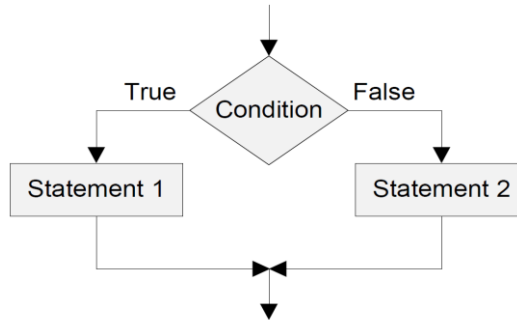
Sequence Structure

Three Control Structures

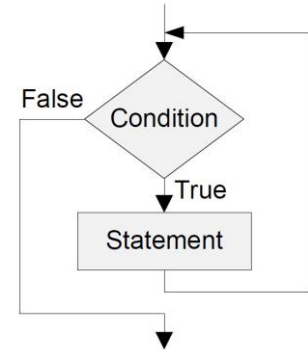
- **Selection Structure**
 - If-then-else
- **Loop Structure**
 - while loop
 - for loop



Sequence Structure



Selection Structure



Loop Structure

Unconditional branching

b Label

label? What is that? Branch instructions use immediate values, but these values would need to be manually calculated by hand. To get around this, we use labels. The assembler will read the labels and automatically know what immediate value to apply to the branch instruction. Example:

```
b jump_here
```

```
add R1, R2, R3
```

```
jump_here:  
mov R1, #1
```

Unconditional branching

Example:

```
MOVS r1, #1
B target ; Branch to target
MOVS r2, #2 ; Not executed
MOVS r3, #3 ; Not executed
MOVS r4, #4 ; Not executed
target MOVS r5, #5
```

- A **label** marks the location of an instruction
- Labels helps humans to read the code
- In machine program, labels are converted to numeric offsets by the assembler

Unconditional Branch Instructions

Instruction	Operands	Brief description
B	label	Branch
BL	label	Branch with Link
BLX	Rm	Branch indirect with Link
BX	Rm	Branch indirect

- **B label**
 - cause a branch to label
- **BL label**
 - copy the address of the next instruction into r14 (lr, the link register), and cause a branch to label
- **BX Rm**
 - branch to the address held in Rm
- **BLX Rm:**
 - copy the address of the next instruction into r14 (lr, the link register) and branch to the address held in Rm

Conditional branching

beq Label

The processor will *branch* to the label if the conditional branch is true. For this we need to make a *comparison*

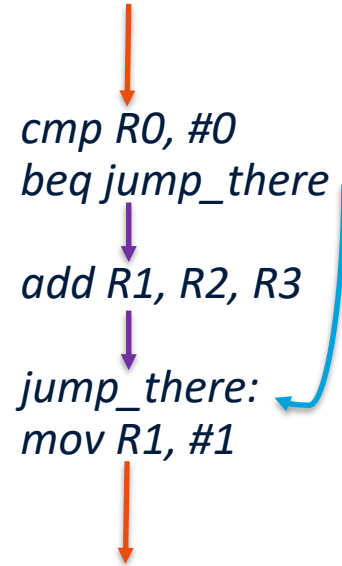
CMP Rn, Operand2

CMN Rn, Operand2

- Update N, Z, C and V according to the result
- **CMP** *subtracts* Operand2 from Rn
 - Same as SUBS, except result is discarded
- **CMN** *adds* Operand2 to Rn
 - Same as ADDS, except result is discarded

Conditional branching

Example:



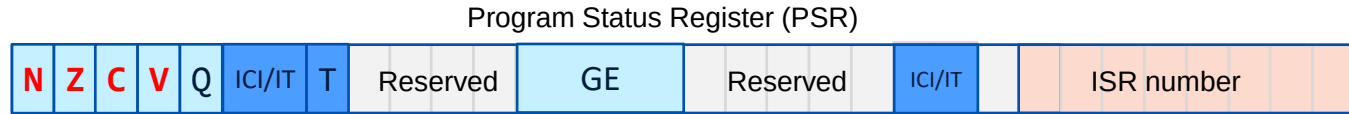
Updating Condition Flags: TST and TEQ

TST Rn, Operand2 ; Bitwise AND

TEQ Rn, Operand2 ; Bitwise Exclusive OR

- Update N and Z according to the result
- Can update C during the calculation of Operand2
- Do not affect V
- TST performs bitwise AND on Rn and Operand2.
 - Same as ANDS, except result is discarded.
- TEQ performs bitwise Exclusive OR on Rn and Operand2.
 - Same as EORS, except result is discarded.

Review: Condition Flags



- **Negative** bit
 - N = 1 if most significant bit of result is 1
- **Zero** bit
 - Z = 1 if all bits of result are 0
- **Carry** bit
 - For unsigned addition, C = 1 if carry takes place
 - For unsigned subtraction, C = 0 (carry = not borrow) if borrow takes place
 - For shift/rotation, C = last bit shifted out
- **oVerflow** bit
 - V = 1 if adding 2 same-signed numbers produces a result with the opposite sign
 - Positive + Positive = Negative, or
 - Negative + negative = Positive
 - Non-arithmetic operations does not touch V bit, such as MOV, AND, LSL, MUL

Updating Condition Flags

- Method 1: append with “S”
 - `ADD r0,r1,r2` → `ADDS r0,r1,r2`
 - `SUB r0,r1,r2` → `SUBS r0,r1,r2`
- Method 2: using compare instructions

Instruction	Brief description	Flags
CMP	Compare	N,Z,C,V
CMN	Compare Negative	N,Z,C,V
TEQ	Test Equivalence	N,Z,C
TST	Test	N,Z,C

Updating Condition Flags

Instruction	Operands	Brief description	Flags
CMP	Rn, Op2	Compare	N,Z,C,V
CMN	Rn, Op2	Compare Negative	N,Z,C,V
TEQ	Rn, Op2	Test Equivalence	N,Z,C
TST	Rn, Op2	Test	N,Z,C

➤ Update the status flags

- No need to add S bit.
- No need to specify destination register.

➤ Operations are:

- **CMP** operand1 - operand2, but result not written
- **CMN** operand1 + operand2, but result not written
- **TST** operand1 & operand2, but result not written
- **TEQ** operand1 ^ operand2, but result not written

➤ Examples:

- **CMP** r0, r1
- **TST** r2, #5

Condition Codes

Suffix	Description
EQ	EQual
NE	Not EQual
CS/HS	Unsigned Higher or Same
CC/LO	Unsigned Lower
MI	Minus (Negative)
PL	Plus (Positive or Zero)
VS	oVerflow Set
VC	oVerflow Clear
HI	Unsigned Higher
LS	Unsigned Lower or Same
GE	Signed Greater or Equal
LT	Signed Less Than
GT	Signed Greater Than
LE	Signed Less than or Equal
AL	ALways

Note AL is the default and does not need to be specified

Signed vs. Unsigned Comparison

Conditional codes applied to
branch instructions

Compare	Signed	Unsigned
>	GT	HI
≥	GE	HS
<	LT	LO
≤	LE	LS
==	EQ	
≠	NE	



Compare	Signed	Unsigned
>	BGT	BHI
≥	BGE	BHS
<	BLT	BLO
≤	BLE	BLS
==	BEQ	
!=	BNE	

Branch Instructions

	Instruction	Description	Flags tested
Unconditional Branch	B <i>Label</i>	Branch to label	
Conditional Branch	BEQ <i>Label</i>	Branch if E Qual	Z = 1
	BNE <i>Label</i>	Branch if N ot E qual	Z = 0
	BCS/BHS <i>Label</i>	Branch if unsigned H igher or S ame	C = 1
	BCC/BLO <i>Label</i>	Branch if unsigned L ower	C = 0
	BMI <i>Label</i>	Branch if M inus (Negative)	N = 1
	BPL <i>Label</i>	Branch if P lus (Positive or Zero)	N = 0
	BVS <i>Label</i>	Branch if o Verflow S et	V = 1
	BVC <i>Label</i>	Branch if o Verflow C lear	V = 0
	BHI <i>Label</i>	Branch if unsigned H igher	C = 1 & Z = 0
	BLS <i>Label</i>	Branch if unsigned L ower or S ame	C = 0 or Z = 1
	BGE <i>Label</i>	Branch if signed G reater or E qual	N = V
	BLT <i>Label</i>	Branch if signed L ess T han	N != V
	BGT <i>Label</i>	Branch if signed G reater T han	Z = 0 & N = V
	BLE <i>Label</i>	Branch if signed L ess than or E qual	Z = 1 or N = ! V

Signed Greater or Equal ($N == V$)

CMP r0, r1

perform subtraction $r0 - r1$, without saving the result

	$N = 0$	$N = 1$
$V = 0$	- No overflow, implying the result is correct. - The result is non-negative, - Thus $r0 - r1 \geq 0$, i.e., $r0 \geq r1$	- No overflow, implying the result is correct. - The result is negative. - Thus $r0 - r1 < 0$, i.e., $r0 < r1$
$V = 1$	- Overflow occurs, implying the result is incorrect. - The result is mistakenly reported as non-negative and in fact it should be negative. - Thus $r0 - r1 < 0$ in reality, i.e., $r0 < r1$	- Overflow occurs, implying the result is incorrect. - The result is mistakenly reported as negative and in fact it should be non-negative. - Thus $r0 - r1 \geq 0$ in reality, i.e. $r0 \geq r1$

Conclusions:

- If $N == V$, then it is signed greater or equal (GE).
- Otherwise, it is signed less than (LT)

Signed Greater or Equal ($N == V$)

CMP r0, r1

perform subtraction $r0 - r1$, without saving the result

	$N = 0$	$N = 1$
$V = 0$	$r0 \geq r1$	$r0 < r1$
$V = 1$	$r0 < r1$	$r0 \geq r1$

Conclusions:

- If $N == V$, then it is signed greater or equal (GE).
- Otherwise, it is signed less than (LT)

Signed Greater or Equal ($N == V$)

CMP r0, r1

perform subtraction $r0 - r1$, without saving the result

	$N = 0$	$N = 1$
$V = 0$	1	0
$V = 1$	0	1

Conclusions:

- If $N == V$, then it is signed greater or equal (GE).
- Otherwise, it is signed less than (LT)

Number Interpretation

Which is greater?

0xFFFFFFFF or **0x00000001**

- If they represent signed numbers, the latter is greater.
(**1** > **-1**).
- If they represent unsigned numbers, the former is greater.
(**4294967295** > **1**).

Which is Greater: 0xFFFFFFFF or 0x00000001?

It's **software's responsibility** to tell computer how to interpret data:

- If written in C, declare the signed vs unsigned variable
- If written in Assembly, use signed vs unsigned branch instructions

```
signed int x, y ;  
x = -1;  
y = 1;  
if (x > y)  
    ...
```

```
MOVS r5, #0xFFFFFFFF  
MOVS r6, #0x00000001  
CMP  r5, r6  
BLE Then_Clause  
...
```

BLE: Branch if less than or equal, signed $\leq$

```
unsigned int x, y ;  
x = 4294967295;  
y = 1;  
if (x > y)  
    ...
```

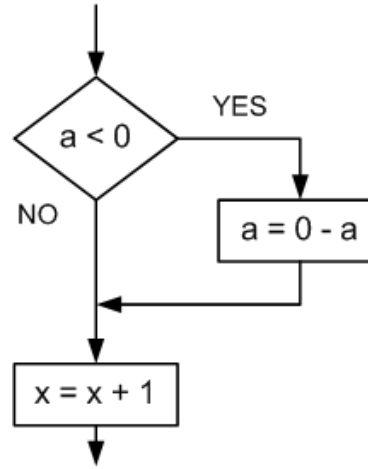
```
MOVS r5, #0xFFFFFFFF  
MOVS r6, #0x00000001  
CMP  r5, r6  
BLS Then_Clause  
...
```

BLS: Branch if lower or same, unsigned $\leq$

If-then Statement

C Program

```
// a is signed integer
if (a < 0 ) {
    a = 0 - a;
}
x = x + 1;
```



Implementation 1:

```
    ; r1 = a (signed integer), r2 = x
    CMP r1, #0          ; Compare a with 0
    BGE endif          ; Go to endif if a ≥ 0
then  RSB r1, r1, #0    ; a = - a
endif ADD r2, r2, #1    ; x = x + 1
```

Reverse Subtract
r1 = 0 - r1

Compound Boolean Expression

C Program	Assembly Program
<pre>// x is a signed integer if(x > 20 && x <= 25) { a = 1 }</pre>	<pre>CMP r0, #20 // compare x and 20 BLE endif // go to then if x ≤ 20 CMP r0, #25 // compare x and 25 BGT endif // go to endif if x > 25 MOV r1, #1 // a = 1 endif:</pre>

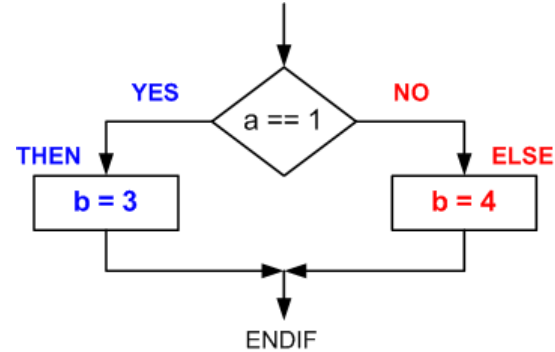
Compound Boolean Expression

C Program	Assembly Program
<pre>// x is a signed integer if(x <= 20 x >= 25){ a = 1; }</pre>	<pre> ; r0 = x CMP r0, #20 ; compare x and 20 BLE then ; go to then if x ≤ 20 CMP r0, #25 ; compare x and 25 BLT endif ; go to endif if x < 25 then MOV r1, #1 ; a = 1 endif</pre>

If-then-else

C Program

```
if (a == 1)
    b = 3;
else
    b = 4;
```

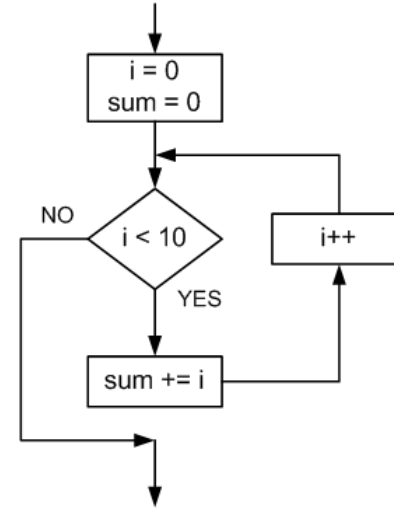


```
        ; r1 = a, r2 = b
        CMP r1, #1      ; compare a and 1
        BNE else        ; go to else if a ≠ 1
then      MOV r2, #3     ; b = 3
          B   endif      ; go to endif
else      MOV r2, #4     ; b = 4
endif
```

For Loop

C Program

```
int i;  
int sum = 0;  
for(i = 0; i < 10; i++){  
    sum += i;  
}
```



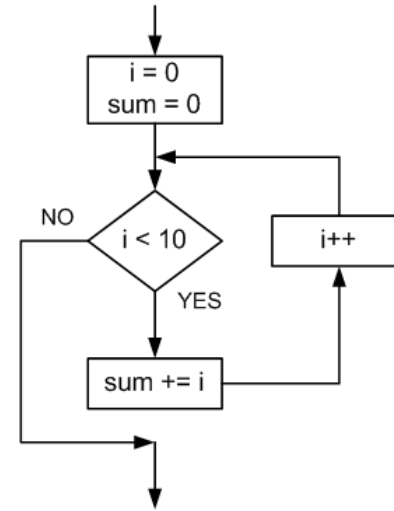
Implementation 1:

```
MOV r0, #0 ; i  
MOV r1, #0 ; sum  
  
B check  
  
loop ADD r1, r1, r0 ; sum += i  
ADD r0, r0, #1 ; i++  
check CMP r0, #10 ; check whether i < 10  
BLT loop ; loop if signed less than  
endloop
```

For Loop

C Program

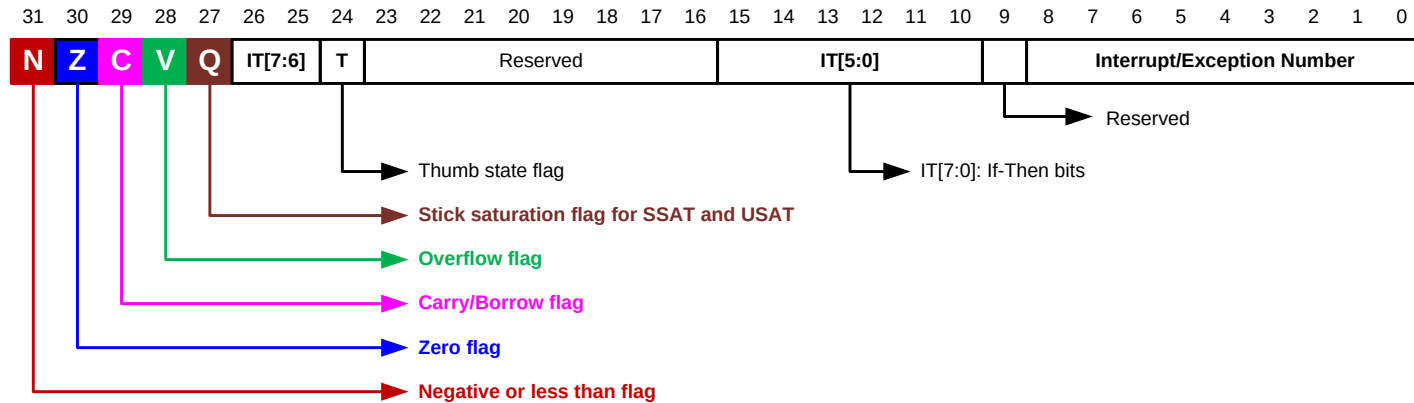
```
int i;  
int sum = 0;  
for(i = 0; i < 10; i++){  
    sum += i;  
}
```



Implementation 2:

```
        MOV r0, #0    ; i  
        MOV r1, #0    ; sum  
  
loop     CMP r0, #10   ; check whether i < 10  
         BGE endloop  ; skip if ≥  
         ADD r1, r1, r0 ; sum += i  
         ADD r0, r0, #1 ; i++  
         B  loop  
  
endloop
```

Combined Program Status Registers (xPSR)



Branching problems

Conditional branching can introduce several issues that affect performance and efficiency:

- Pipeline Stalls & Branch Penalties. Conditional branches introduce uncertainty in pipeline instruction flow. If the processor mispredicts the branch outcome, it has to flush the pipeline and fetch new instructions, leading to delays.
- Increased Execution Time. Since branches can disrupt instruction **prefetching** and **decoding**, they can increase execution cycles. This is especially problematic in real-time embedded systems where deterministic execution is crucial.

Branching problems

- **Branch stall.** A **pipeline stall** occurs when the processor cannot proceed to the next instruction due to dependencies or delays in instruction execution. This results in wasted cycles, reducing overall performance.
- **Data Hazards (Data Dependencies).** If an instruction depends on the result of a previous instruction that has not yet completed, the processor must stall until the result is available. Example:

LDR R1, [R0] ; Load value from memory into R1
ADD R2, R1, #1 ; Use R1 before it is fully loaded

The processor must wait for the LDR instruction to complete before executing ADD, leading to a stall.

Conditional Execution

Conditional execution allows instructions to be executed **instead of branching**, based on flag conditions, reducing branch instructions and improving efficiency

```
if (a <= 0)
    y = -1;
else
    y = 1;
```



```
CMP    r0, #0
MOVLE  r1, #-1 ; executed if LE
MOVGTE r1, #1  ; executed if GT
```

LE: Signed Less than or Equal

GT: Signed Greater Than

Conditional Execution

Conditional execution allows instructions to be executed **instead of branching**, based on flag conditions, reducing branch instructions and improving efficiency

- In ARM (A32), most instructions support condition codes (MOVLE, MOVGT, etc.)
- In Thumb-2 (T32), conditional execution must be inside an IT (If-Then) block
- The **IT instruction** sets conditions for up to 4 following instructions
- Reduces the need for branches, improving pipeline efficiency.

```
CMP    r0, #0      // Compare r0 with 0
IT     LE          // If Less than or Equal...
MOVLE  r1, #-1     // Move -1 to r1
IT     GT          // If Greater than...
MOVGT  r1, #1      ; Move 1 to r1
```


Conditional Execution – IT (IF – THEN)

With IT, you can specify a condition code for up to 4 instructions. For each instruction, you specify if it's part of the If (T) or Else (E).

*The assembler will
complain if the
conditions do not
follow the IT codes*



```
ITTET EQ  
ADDEQ r0,r0,r0  
ADDEQ r1,r0,r0  
ADDNE r2,r0,r0  
ADDEQ r3,r0,r0
```

Conditional Execution

```
if (a <= 0)
    y = -1;
else
    y = 1;
```



a → r0
y → r1

```
CMP    r0, #0
ITE LE
MOVLE  r1, #-1 ; executed if LE
MOVGTE r1, #1  ; executed if GT
```

LE: Signed Less than or Equal

GT: Signed Greater Than

Conditional Execution

```
if (a==1 || a==7 || a==11)
    y = 1;
else
    y = -1;
```



a → r0
y → r1

```
CMP    r0, #1
ITTET  NE
CMPNE  r0, #7 ; executed if r0 != 1
CMPNE  r0, #11 ; executed if r0 != 7
MOVEQ  r1, #1
MOVNE  r1, #-1
```

NE: Not Equal

EQ: Equal

Compound Boolean Expression

C Program	Assembly Program
<pre>// x is a signed integer if(x <= 20 x >= 25){ a = 1; }</pre>	<pre>; r0 = x, r1 = a CMP r0, #20 ; compare x and 20 IT LE MOVLE r1, #1 ; a=1 if less or equal CMP r0, #25 ; CMP if greater than IT GE MOVGE r1, #1 ; a=1 if greater or equal Endif</pre>

Greatest Common Divider (GCD)

What is GCD: The largest positive integer that divides two numbers **a** and **b** without leaving a remainder.

Example:

$$\text{GCD}(48, 18) = 6$$

Greatest Common Divider (GCD)

Euclid's Algorithm: Instead of factoring numbers, use division and remainder to find GCD efficiently.

Steps:

- If $b == 0$, return a (GCD is found).
- Otherwise, replace (a, b) with $(b, a \bmod b)$ and repeat.

$$GCD(a, b) = GCD(b, a \bmod b)$$

$78 \div 66 = 1 \text{ remainder } 12$ ($78 = 66 \times 1 + 12$)

$66 \div 12 = 5 \text{ remainder } 6$ ($66 = 12 \times 5 + 6$)

$12 \div 6 = 2 \text{ remainder } 0$ ($12 = 6 \times 2 + 0$)

6 = Greatest Common Factor

Example 1: Greatest Common Divider (GCD)

Euclid's Algorithm

```
mov r0, #78
mov r1, #66
```

GCDloop:

```
    UDIV    r3, r0, r1    // r3 = a / b (quotient)
    MUL     r3, r3, r1    // r3 = (a / b) * b
    SUB     r2, r0, r3    // r2 = a - (quotient * b) → a % b
    MOV     r0, r1        // a = b
    MOV     r1, r2        // b = a % b
    CMP     r1, #0        // Check if remainder is zero
    BNE     GCDloop       // If not, repeat loop
```

*integer divide instruction computes
(numerator ÷ denominator) and delivers
the quotient*